

DECISION TREES

Entropy • Information Gain • Gini Index • Gain Ratio

A concept-first, worked-example study guide built around the classic
"Should I Play Tennis?" dataset

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
...	...	...	...	... (14 rows total)

Prepared for: **Aadit** | Focus: Backend Engineering & AI/ML, Darshan University

Explains not just what each concept is, but why it exists and how it is computed — worked from first principles on real numbers.

Contents

The Dataset We Will Use

0

The 14-row PlayTennis table — our running example for every concept below.

How the Splitting / Plotting Works

1

Why decision boundaries are always straight lines drawn parallel to one axis at a time.

Advantages & Disadvantages

2

What decision trees are genuinely good at, and where they break down.

Entropy

3

Measuring how 'mixed up' a set of examples is, borrowed from information theory.

Information Gain

4

How the tree decides which feature to split on at every node (the ID3 rule).

Gini Index

5

A faster alternative to entropy, used by the CART algorithm.

Gain Ratio ("Gini Ratio")

6

Fixing Information Gain's bias towards attributes with many values.

0 The Dataset We Will Use

Every concept in this guide — entropy, information gain, Gini index and gain ratio — is computed directly on this same 14-row dataset, so the numbers stay connected and comparable throughout. The goal is to decide, from four weather attributes, whether tennis will be played.

#	Outlook	Temperature	Humidity	Windy	PlayTennis
1	Sunny	Hot	High	False	No
2	Sunny	Hot	High	True	No
3	Overcast	Hot	High	False	Yes
4	Rainy	Mild	High	False	Yes
5	Rainy	Cool	Normal	False	Yes
6	Rainy	Cool	Normal	True	No
7	Overcast	Cool	Normal	True	Yes
8	Sunny	Mild	High	False	No
9	Sunny	Cool	Normal	False	Yes
10	Rainy	Mild	Normal	False	Yes
11	Sunny	Mild	Normal	True	Yes
12	Overcast	Mild	High	True	Yes
13	Overcast	Hot	Normal	False	Yes
14	Rainy	Mild	High	True	No

14 instances — 9 labelled **Yes**, 5 labelled **No**. This is the classic Mitchell / Quinlan "PlayTennis" dataset.

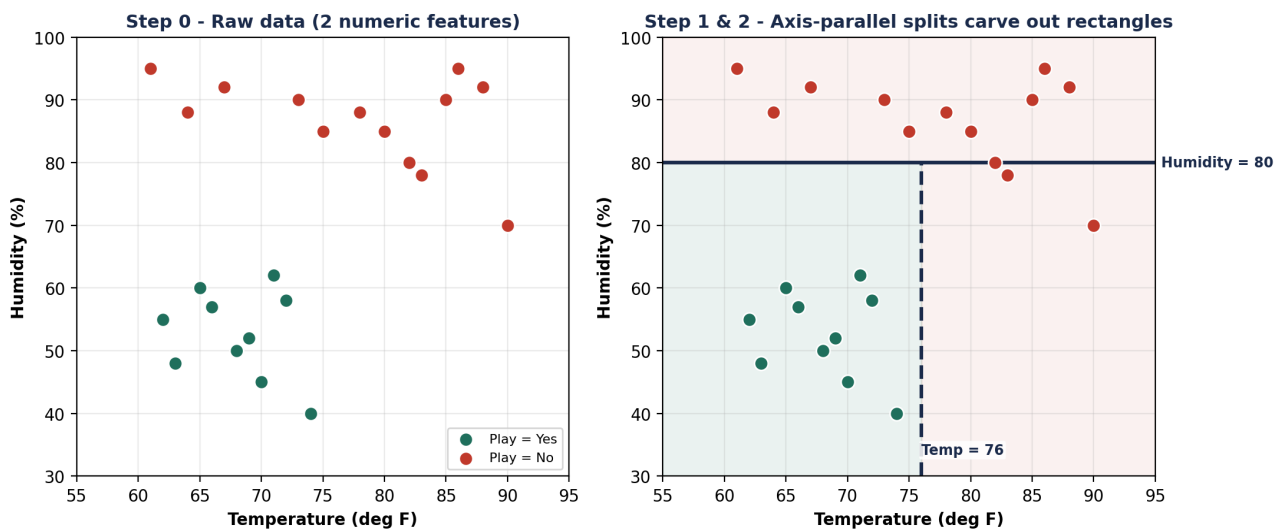
1 How the Splitting / Plotting Works

What it means: at every internal node, a decision tree asks a question about **exactly one** feature — never a combination of features. In the tennis example the root node asks only “What is Outlook?”, not something like “Is Outlook Sunny AND Humidity High?” in a single step.

For a **numeric** feature the question takes the form “Is feature $\leq$ threshold?” (e.g. Temperature $\leq$ 76). Geometrically, this single condition is a straight line that is **perpendicular to the Temperature axis** — which is the same as saying it runs **parallel to every other axis**. This is exactly what people mean when they say decision tree boundaries are “lines parallel to an axis.” For a **categorical** feature (like Outlook or Humidity in our table) the same idea holds: the node fans out into one branch per category, each branch keeping every other feature untouched.

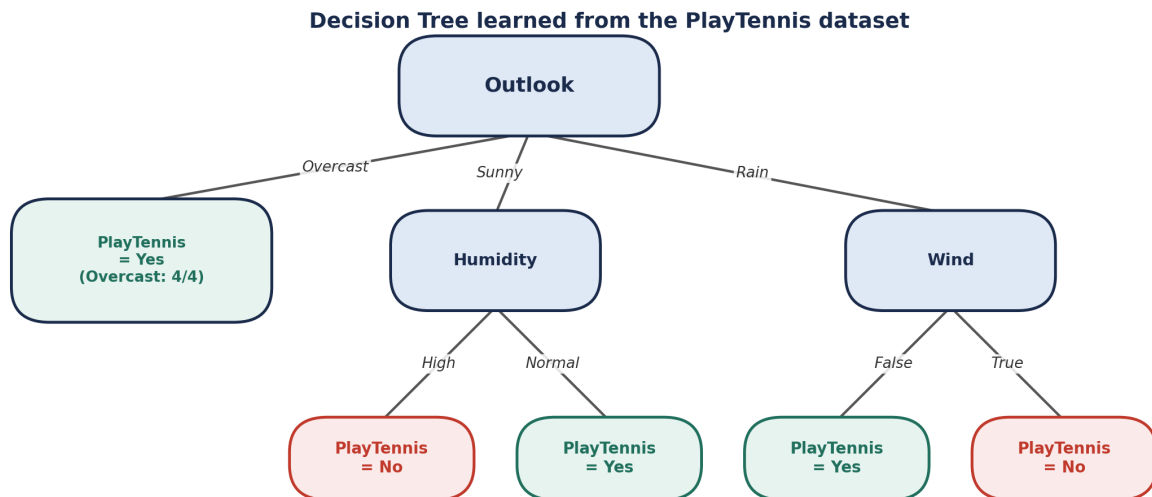
■ Why is it always axis-parallel, never diagonal?

Because a single split rule only ever looks at **one column** of the table at a time. A diagonal boundary would require a rule like “ $0.7 \times \text{Temperature} + 0.3 \times \text{Humidity} \leq 60$ ”, which mixes two features together — and a plain decision tree’s split test has no mechanism to do that. Each split can only cut the feature space along one dimension, so the boundaries it produces are always **rectangles (in 2D)** or **hyper-rectangles (in higher dimensions)**, built one straight cut at a time.



Illustrative numeric example (not the tennis data) showing why splits look like straight lines. First cut: a horizontal line at Humidity = 80 (parallel to the Temperature axis). Second cut: inside the lower region, a vertical line at Temperature = 76 (parallel to the Humidity axis). Two straight, axis-parallel cuts already isolate a clean rectangular 'Yes' region.

How this looks with our actual (categorical) tennis data: instead of numeric thresholds, each split fans out into branches, but the underlying idea is identical — one feature is “sliced” at a time, and every branch inherits all the previous constraints. Splitting on Outlook first divides the 14 rows into three separate groups (Sunny / Overcast / Rainy). Splitting again inside the Sunny group on Humidity divides those 5 rows further, without ever touching the rows that went into the Overcast or Rainy groups. The result, shown below, is the full tree — each root-to-leaf path is one “axis-parallel cut” after another, together carving the original 14 rows into pure, non-overlapping regions.



The tree built from the PlayTennis table using the splitting logic explained in Sections 3-4.

■ Why does this matter in practice?

Because splits are always axis-parallel, a decision tree can perfectly separate data that forms “staircase” or rectangular patterns with very few cuts, but it can only **approximate** a diagonal or curved boundary by stacking many tiny rectangular steps — which needs a deeper tree and more data. This directly explains one of the disadvantages you’ll see in the next section: trees can become unnecessarily large when the true decision boundary is not aligned with the axes.

2 Advantages & Disadvantages

Advantages

- **Easy to interpret** — The tree in Section 1 can be read out loud as plain if/else rules (e.g. “If Outlook is Sunny and Humidity is High, don't play”). No statistics background is needed to explain a prediction.
- **Little data preparation** — Works directly on the raw Outlook / Temperature / Humidity / Windy columns. No need to scale numbers or centre them like for distance-based or gradient-based models.
- **Handles mixed data types** — Categorical columns (Outlook, Windy) and numeric columns can sit in the same tree without special encoding.
- **Captures non-linear patterns** — Because it splits repeatedly, it can model interactions like “Outlook matters only when combined with Humidity” automatically, without the analyst specifying that interaction.
- **Built-in feature ranking** — Information Gain / Gini Gain (Sections 4-5) naturally rank Outlook > Humidity > Wind > Temperature by importance.
- **Fast to train and to predict** — Prediction is just following a short root-to-leaf path — a handful of comparisons.

Disadvantages

- **Overfitting** — A tree grown fully (splitting until every leaf is pure) can memorise the training rows — including noise — and generalise poorly. With only 14 rows, a tree could easily grow one branch per row.
- **High variance / instability** — Small changes in the data can pick a different root attribute (Section 4), producing a noticeably different tree — decision trees are known to be a “high-variance” model.
- **Axis-parallel bias** — As shown in Section 1, splits are always parallel to an axis, so diagonal or curved true boundaries need many extra splits to approximate, growing the tree unnecessarily.
- **Greedy, not globally optimal** — Information Gain / Gini are computed one split at a time (“greedy”). The attribute that looks best right now is not guaranteed to lead to the smallest or most accurate overall tree.
- **Bias toward many-valued attributes** — Raw Information Gain favours attributes with many distinct categories (covered in Section 6) — which can pick a useless, over-specific attribute if left uncorrected.
- **Can be less accurate alone** — A single tree usually underperforms ensemble methods (Random Forests, Gradient Boosted Trees) that combine many trees to reduce variance.

■ Why learn Entropy, Information Gain, Gini Index and Gain Ratio at all?

*Every advantage above (interpretability, ranking features, capturing patterns) and every fix for the disadvantages (pruning, choosing splits sensibly) depends on having a **numeric score** for “how good is this split?”. That score is exactly what the next four sections build, one piece at a time: Entropy measures impurity → Information Gain measures the drop in impurity from a split → Gini Index is a faster alternative impurity measure → Gain Ratio fixes Information Gain's bias.*

3 Entropy

What it is: Entropy is a single number that measures how **mixed up (impure)** the class labels are inside a set of examples. It comes from Claude Shannon's Information Theory, where it originally measured the expected number of bits needed to encode/identify an outcome.

■ Why do we need it at all?

To build a tree we must repeatedly answer the question: “Is this node's data still a jumbled mix of Yes/No, or is it mostly one class?” A plain count of Yes vs No isn't enough on its own to compare across nodes of different sizes and different proportions. Entropy turns “how mixed” into one comparable number between 0 and 1 (for two classes), which the tree can then use to search for the split that reduces the mixing the most (that search is Information Gain — Section 4).

How it is calculated:

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

S = the set of examples at the node | p_i = proportion of examples belonging to class i

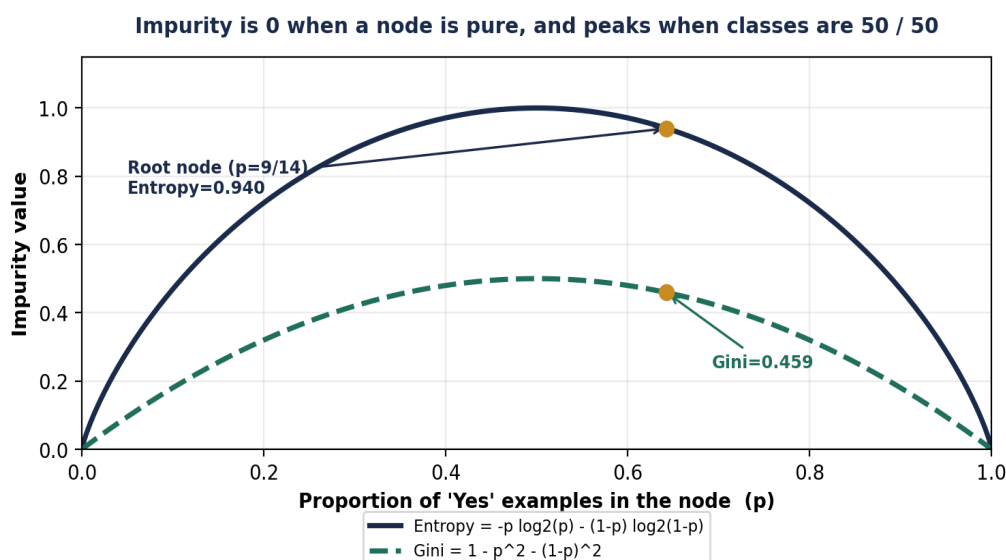
In our binary Yes/No problem this expands to just two terms:

$$\text{Entropy}(S) = -p_{\text{Yes}} \log_2(p_{\text{Yes}}) - p_{\text{No}} \log_2(p_{\text{No}})$$

Worked example — the root node (all 14 rows):

The full table has 9 “Yes” rows and 5 “No” rows, so $p_{\text{Yes}} = 9/14 = 0.643$ and $p_{\text{No}} = 5/14 = 0.357$.

$$\begin{aligned} \text{Entropy}(S) &= -(9/14) \times \log_2(9/14) - (5/14) \times \log_2(5/14) \\ &= -(0.643) \times (-0.637) - (0.357) \times (-1.485) \\ &= 0.410 + 0.530 \\ &= \mathbf{0.940} \end{aligned}$$



Entropy (solid) peaks at 1.0 when a node is a perfect 50/50 mix, and falls to 0 when a node is pure. Our root node ($p = 9/14 \approx 0.64$) sits close to the peak at 0.940 — confirming the root is fairly mixed, which is exactly why splitting it helps.

■ How to read the number 0.940

Entropy of **0** means the node is perfectly pure (every row has the same label — no uncertainty at all).
Entropy of **1** (for two classes) means the node is a perfect 50/50 coin-flip — maximum uncertainty.
Our root's 0.940 is close to 1, telling us the root node is still highly mixed and a good split is needed before we can make a confident prediction.

4 Information Gain

What it is: Information Gain measures **how much the entropy drops** after splitting a node on a chosen attribute. It is literally “entropy before the split” minus “the (weighted) average entropy after the split.”

■ Why do we need it, if we already have Entropy?

Entropy alone only scores **one** set of examples. It cannot, by itself, tell us **which of the four attributes** (Outlook, Temperature, Humidity, Windy) is the best one to split on next. Information Gain answers exactly that: it tries splitting on each candidate attribute, measures how much each split lowers the average entropy of the resulting branches, and the tree simply **picks the attribute with the highest gain**. This greedy “pick the best gain at each node” procedure is the ID3 algorithm.

How it is calculated:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{values}(A)} (|S_v| / |S|) \times \text{Entropy}(S_v)$$

A = the attribute being tested | S_v = the subset of S where attribute A has value v

Worked example — evaluating “Outlook” at the root: Outlook splits the 14 rows into three branches. We first compute the entropy of each branch on its own.

Outlook value	Rows	Yes	No	Entropy of branch
Sunny	5	2	3	0.971
Overcast	4	4	0	0.000 (pure!)
Rainy	5	3	2	0.971

Now take the weighted average of these three branch entropies, and subtract from the root's entropy (0.940):

$$\text{Weighted avg. entropy} = (5/14)(0.971) + (4/14)(0.000) + (5/14)(0.971) = 0.347 + 0 + 0.347 = 0.694$$

$$\text{Gain}(S, \text{Outlook}) = 0.940 - 0.694 = \mathbf{0.246}$$

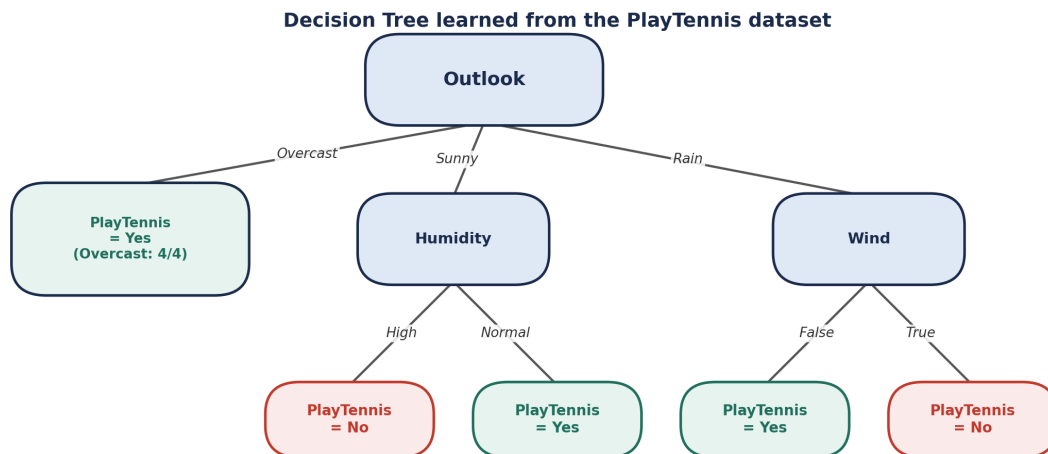
Repeating the exact same three steps (branch entropies → weighted average → subtract from 0.940) for the remaining three attributes gives:

Attribute	Weighted avg. entropy after split	Information Gain
Outlook	0.694	0.246
Humidity	0.788	0.152
Windy	0.892	0.048
Temperature	0.911	0.029

Outlook has the highest Information Gain, so ID3 selects it as the root of the tree.

■ Why does splitting continue after the root?

The same procedure is simply repeated **inside each branch** on the remaining rows and remaining attributes. Inside the Sunny branch (5 rows, entropy 0.971), testing Humidity gives a perfect split — High → all 3 rows are No (entropy 0), Normal → both rows are Yes (entropy 0) — so $\text{Gain}(\text{Sunny}, \text{Humidity}) = 0.971 - 0 = 0.971$, the maximum possible. The same happens inside the Rainy branch with Windy. The Overcast branch is already pure (entropy 0), so it needs no further splitting and becomes a leaf immediately. This is exactly the tree shown back in Section 1.



Final tree: at every node, the attribute with the highest Information Gain among the remaining attributes was chosen.

5 Gini Index

What it is: The Gini Index is a second way to measure impurity — an alternative to Entropy. It measures the probability of **misclassifying** a randomly chosen example from the node, if you labelled it randomly according to the class proportions already present in that node. It is the impurity measure used by the **CART** algorithm (the basis of scikit-learn's DecisionTreeClassifier).

■ Why do we need a second impurity measure — isn't Entropy enough?

*Entropy works well but its formula needs a **logarithm** for every class, at every candidate split, at every node — which is comparatively expensive to compute, especially on large datasets. Gini Index measures almost the same thing (how mixed a node is) using only **squares**, which are much cheaper to compute. In practice Entropy and Gini nearly always agree on which split is best (as they do in our tennis example), so Gini is often preferred purely for speed, not for better accuracy.*

How it is calculated:

$$\text{Gini}(S) = 1 - \sum p_i^2$$

Same p_i as before: the proportion of examples in class i

Worked example — the root node:

$$\begin{aligned}\text{Gini}(S) &= 1 - (9/14)^2 - (5/14)^2 \\ &= 1 - 0.413 - 0.128 \\ &= \mathbf{0.459}\end{aligned}$$

Just like Information Gain, we can measure the **Gini Gain** of an attribute: the drop in Gini Index after splitting, weighted by branch size. Applying this to all four attributes:

Attribute	Weighted avg. Gini after split	Gini Gain (drop from 0.459)
Outlook	0.343	0.116
Humidity	0.367	0.092
Windy	0.429	0.031
Temperature	0.440	0.019

Gini Index ranks the attributes in exactly the same order as Information Gain did: Outlook > Humidity > Windy > Temperature.

Entropy and **Gini Index** side by side, both reaching the same conclusion (Outlook wins):

Attribute	Info. Gain (Entropy-based)	Gini Gain	Agree on ranking?
Outlook	0.246 (highest)	0.116 (highest)	Yes
Humidity	0.152	0.092	Yes
Windy	0.048	0.031	Yes
Temperature	0.029 (lowest)	0.019 (lowest)	Yes

■ Entropy vs Gini — the practical difference

Look back at the curve diagram in Section 3: the Gini curve is slightly “flatter” near the edges than the Entropy curve, meaning Gini is a little less sensitive to changes in very pure nodes, while Entropy penalises impurity slightly more sharply. In practice this rarely changes the resulting tree by much. **ID3** and **C4.5** use Entropy + Information Gain (or Gain Ratio, Section 6); **CART** uses the Gini Index because it is cheaper to compute and also extends naturally to regression trees (via variance) as well as classification.

6 Gain Ratio (“Gini Ratio”)

■ A quick naming clarification first

There isn't a standard measure literally called “Gini Ratio.” The two real, distinct concepts are the **Gini Index** (Section 5, an impurity measure) and the **Gain Ratio** (this section, a corrected version of Information Gain). They are easy to mix up because both names contain “Gini/Gain,” but they solve different problems. This section covers Gain Ratio, since that is the metric that naturally follows Sections 3-5 as the standard next concept in the ID3 → C4.5 storyline.

What it is: Gain Ratio is a normalised version of Information Gain, introduced in the **C4.5** algorithm (the successor to ID3) specifically to correct a weakness of raw Information Gain.

■ Why plain Information Gain isn't always safe to use

Information Gain quietly favours attributes that split the data into **many small branches**. As an extreme example, imagine we had added a “Day” column with 14 unique values (Day1...Day14) to our table. Splitting on Day would create 14 branches, each with a single row — every branch would be perfectly pure (entropy 0), giving Day the maximum possible Information Gain of 0.940! Yet this split is **useless**: it has memorised the training rows instead of learning a general pattern, and it cannot predict anything sensible for a new, unseen day. Even within our real attributes, **Outlook** (3 branches) has a slight structural advantage over **Humidity** (only 2 branches) simply because it splits the data into more, smaller pieces — independent of how genuinely useful that split is. Gain Ratio corrects for this by dividing Gain by a penalty term that grows with the **number and evenness** of an attribute's branches.

How it is calculated:

First compute **Split Information** — this measures how many branches an attribute creates and how evenly the data is spread across them (notice it uses the branch sizes only, not the class labels at all):

$$\text{SplitInfo}(S, A) = -\sum_v (|S_v| / |S|) \times \log_2(|S_v| / |S|)$$

Then divide the ordinary Information Gain by this Split Information:

$$\text{GainRatio}(S, A) = \text{Gain}(S, A) / \text{SplitInfo}(S, A)$$

Worked example — Split Information of Outlook (3 branches: 5, 4, 5 rows):

$$\begin{aligned} \text{SplitInfo}(\text{Outlook}) &= -(5/14)\log_2(5/14) - (4/14)\log_2(4/14) - (5/14)\log_2(5/14) \\ &= 0.530 + 0.516 + 0.530 = \mathbf{1.577} \end{aligned}$$

$$\text{GainRatio}(\text{Outlook}) = 0.246 / 1.577 = \mathbf{0.156}$$

Repeating for all four attributes:

Attribute	# Branches	Split Info	Info Gain	Gain Ratio
Outlook	3	1.577	0.246	0.156
Humidity	2	1.000	0.152	0.152
Windy	2	0.985	0.048	0.049
Temperature	3	1.556	0.029	0.019

■ What actually changed, and why it matters

Outlook still wins (0.156 vs 0.152), but notice the **gap has shrunk dramatically**: by raw Information Gain, Outlook beat Humidity by $0.246 - 0.152 = 0.094$; by Gain Ratio the margin is only $0.156 - 0.152 = 0.004$, almost a tie. That shrinkage is Gain Ratio doing exactly its job — it detected that Outlook's advantage was partly just because it has an extra branch, and discounted it accordingly. On a dataset containing a useless many-valued column like "Day," this correction would be decisive rather than marginal: Day's huge Split Information ($\log_2 14 \approx 3.807$) would crush its inflated Gain, dropping its Gain Ratio far below Outlook or Humidity. In practice, C4.5 computes Gain Ratio only among attributes whose raw Information Gain is at least the average, so that an attribute with both low gain and low split info doesn't win by a fluke.

Quick Reference — All Four Metrics at the Root Node

Metric	Formula	Root value	Used by	Purpose
Entropy	$-\sum p_i \log_2(p_i)$	0.940	ID3, C4.5	Measures impurity of one node
Information Gain	$\text{Entropy}(S) - \text{weighted avg. Entropy}(S_v)$	0.246 (Outlook)	ID3	Chooses best attribute to split on
Gini Index	$1 - \sum p_i^2$	0.459	CART	Faster alternative impurity measure
Gain Ratio	$\text{Gain}(S,A) / \text{SplitInfo}(S,A)$	0.156 (Outlook)	C4.5	Corrects Gain's many-value bias

Key Takeaways

- Every split a decision tree makes is a straight cut parallel to one feature's axis — never a diagonal — because each rule tests only one attribute at a time.
- Decision trees are prized for interpretability and handling mixed data with little preparation, but are prone to overfitting, instability, and greedy (locally-best) choices.
- Entropy quantifies how mixed a node's classes are; 0 = pure, 1 = maximally mixed (for two balanced classes).
- Information Gain = Entropy before a split minus the weighted average Entropy after — the tree greedily picks the attribute that maximises this at each node.
- Gini Index is a computationally cheaper stand-in for Entropy that almost always agrees on which split is best; it powers the CART algorithm.
- Gain Ratio divides Information Gain by Split Information to stop the tree from being seduced by attributes that merely have many branches, rather than genuinely informative ones.